

本小节内容

冒泡排序实战

冒泡排序实战

上一小节我们讲解了冒泡排序的原理，下面我们来进行代码实战。

代码实战步骤：首先我们通过随机数生成 10 个元素，通过随机数生成，我们可以多次测试排序算法是否正确，然后打印随机生成后的元素顺序，然后通过冒泡排序对元素进行排序，然后再次打印排序后的元素顺序。

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
typedef int ElemType;
typedef struct{
    ElemType *elem;//存储元素的起始地址
    int TableLen;//元素个数
}SSTable;

void ST_Init(SSTable &ST,int len)
{
    ST.TableLen=len;
    ST.elem=(ElemType *)malloc(sizeof(ElemType)*ST.TableLen);//申请一块堆空间，当数组来使用
    int i;
    srand(time(NULL));//随机数生成，每一次执行代码就会得到随机的 10 个元素
    for(i=0;i<ST.TableLen;i++)
    {
        ST.elem[i]=rand()%100;//生成的是 0-99 之间
    }
}
//打印数组中的元素
void ST_print(SSTable ST)
{
    for(int i=0;i<ST.TableLen;i++)
    {
        printf("%3d",ST.elem[i]);
    }
    printf("\n");
}
```

//交换两个元素

```
void swap(ElemType &a, ElemType &b)
{
    ElemType tmp;
    tmp=a;
    a=b;
    b=tmp;
}
```

// 64 94 95 79 69 84 18 22 12 78

// 12 64 94 95 79 69 84 18 22 78 外层完成一次循环后，最小值到了最前面

```
void BubbleSort(ElemType A[], int n)
```

```
{
    int i, j;
    bool flag;
    for(i=0; i<n-1; i++)//i 最多访问到 8
    {
        flag=false;//元素是否发生交换的标志
        for(j=n-1; j>i; j--)//把最小值就放在最前面
        {
            if(A[j-1]>A[j])
            {
                swap(A[j-1], A[j]);
                flag=true;
            }
        }
        if(flag==flag)//如果一趟比较没有发生任何交换，说明有序，提前结束排序
            return;
    }
}
```

//《王道 C 督学营》课程

//冒泡排序

```
int main()
```

```
{
    SSTable ST;
    ST_Init(ST, 10);//初始化
    // ElemType A[10]={ 64, 94, 95, 79, 69, 84, 18, 22, 12, 78};
    //内存 copy 接口, 当你 copy 整型数组, 或者浮点型时, 要用 memcpy, 不能用 strcpy, 初试考 memcpy
    概率很低
    //memcpy(ST.elem, A, sizeof(A));//这是为了降低调试难度，每次数组数据固定而设计的
    ST_print(ST);//排序前打印
    BubbleSort(ST.elem, 10);
    ST_print(ST);//排序后再次打印
    return 0;
}
```

```
}
```

最后我们计算一下时间复杂度与空间复杂度，时间复杂度其实就是程序实际的运行次数，可以看到内层是 $j > i$ ，外层 i 的值是从 0 到 $N-1$ ，所以程序的总运行次数是 $1+2+3+\dots+(N-1)$ ，即从 1 一直加到 $N-1$ ，这是等差数列求和，得到的结果是 $N(N-1)/2$ ，即总计运行了这么多次，忽略了低阶项和高阶项的首项系数，因为时间复杂度为 $O(n^2)$ 。因为未使用额外的空间（额外空间必须与输入元素的个数 N 相关），所以空间复杂为 $O(1)$ 。如果数组本身有序，那么就是最好的时间复杂度 $O(n)$ 。